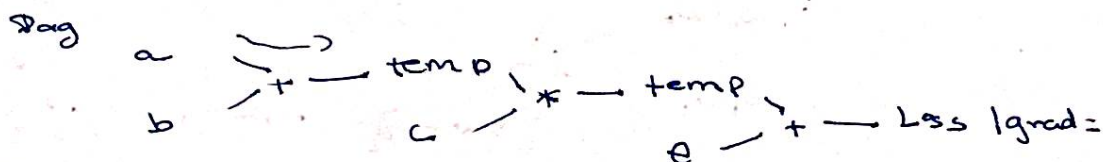


Brain Dump:

First 40 minutes:

- Micrograd is basically an autograd engine that performs back propagation from scratch.
- Also has a DAG built dynamically.

eg: $d = (a+b) * c + e$



- The class/type is value

value: data
prev
grad
op
label

does basic math operation

- Derivative: if i nudge x how much does y change

Eg: $\lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$
 $\downarrow$ normalize

if $f = x \cdot y$

$$f(x+h) = \frac{(x+h) \cdot y - x \cdot y}{h} = \frac{x \cdot y + h \cdot y - x \cdot y}{h}$$

elif $f = x + y$

$$f(x+h) = \frac{x+h+y - x-y}{h} = \frac{h}{h} = 1$$

Just routes the gradient

Chain Rule:

$$\frac{dL}{da} = \frac{dL}{de} \cdot \frac{de}{da} \rightarrow \text{for each use here}$$

+ value if * then other

if + then 1 so cope past the

[after]

Start grad with 0 and back propagate.

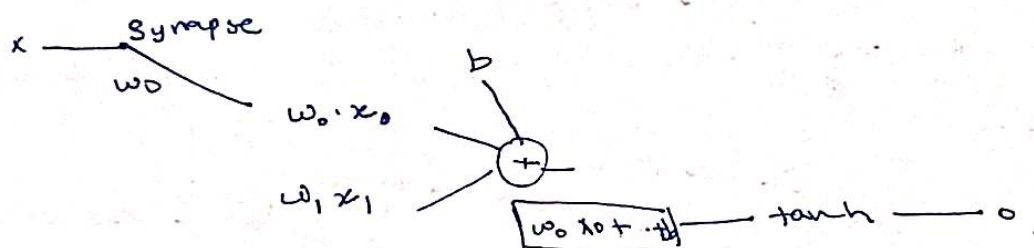
back propagation $\rightarrow$ recursive app. of chain rule backwards in the computation graph.

After finding all the gradients, we ~~add~~ move all the weights toward the gradient

For eg: $a.data += 0.01 * a.grad$

↳ a's val. is moved a little closer toward the direction of gradient.

Neuron



Synaptic
rule:

for pow $\rightarrow$ what is the grad.

$$der = x^2 = 2x \cdot x$$

$$= other * x^{(power-1)} \rightarrow outgrad?$$

all functions $\rightarrow$ has backword func to compute grad.
and its grad should accumulate.

For eg: $a + a = b \mid grad = 1$

$$grad = 2a$$

$$grad = 2a \mid b \mid grad = 1$$

$$\text{since } \frac{db}{da} = d(a+a) = 1+1=2$$

without accumulator $+2$

it becomes 1 since it overrid

Pytorch $\rightarrow$ does it efficiently $\rightarrow$ uses tensors $\rightarrow$ large computations.
we do `.double()` to ~~match~~ match Py float 64
and `grad` $\rightarrow$ requires `grad=True`.

we use `item()` to get the data number, stripping away the tensor.

Eg: `w2.data.item()` or `w2.item` $\rightarrow$ `w2.data`
0.5699 $\rightarrow$ `[0.5699, grad=1]`

Micrograd Second half.

Value class - ✓.

Create neurons $\rightarrow$ layers $\rightarrow$ MLP. classes.

Neuron class:

has w & b initialized.

call func:

does $w_0 x_0 + w_1 x_1 \dots + b$
and return out.

Layer class:

int list
[nin, nouts]

has neurons initialized.

call: outputs neuron val for each neurons.
size nouts

MLP class:

has layer initialized in, out

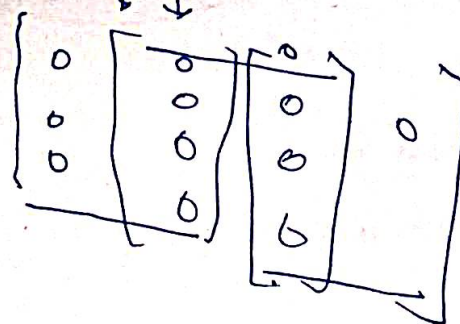
self.layers = [layer(sz[i], sz[i+1]) for i in range(nouts)]

Parameters:

Store w & b

self.w = [b]

sz = [4, 4, 1]



for a dataset

$x_s = \begin{bmatrix} - & - & - & - \\ - & - & - & - \\ - & - & - & - \\ - & - & - & - \end{bmatrix}$

$y_s = [1, -1, 1, -1]$

Loss = $\sum (x_i - y_i)^2$ for x_i, y_i in y_{pred}, y_o

$n_{ypred} = \text{MLP}(3, [4, 4, 1])$

$y_{pred} = [n(x)]$ for x in x_s

Pseudocode:

```
for k in range(20):
```

```
    # forward
```

```
    ypred = [n(x) for x in x2]
```

```
    loss = sum((yout - ygt) ** 2 for ygt, yout in  
                zip(yg, ypred))
```

```
    # backward
```

```
    for p in n.parameters():
```

```
        p.grad = 0.0 → flush the gradients
```

```
    loss.backward()
```

```
    # update.
```

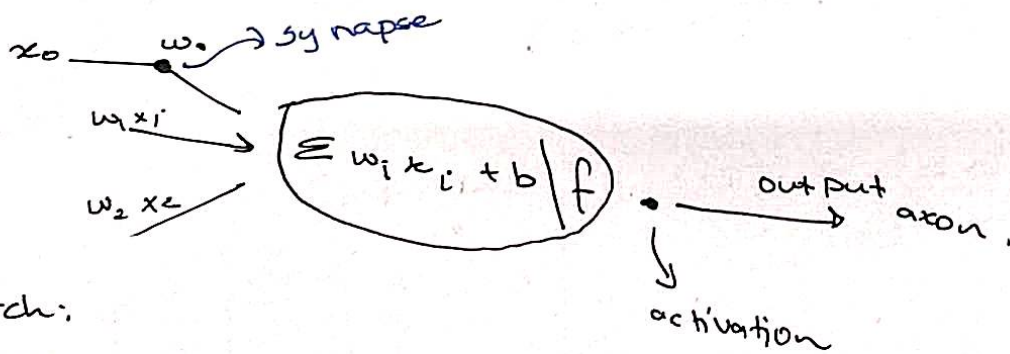
```
    for p in n.parameters():
```

```
        p.data += -0.1 * p.grad
```

```
    print(k, loss.data)
```

we need to bring the loss to 0 → so opp to the gradient.

A neuron:



Pytorch:

Instead of scalar, they have tensors,
for grad: requires_grad=True

It will do its backward. with just a call.

→ they have it all written.